DataScientest / Liora

Classification of Rakuten E-commerce Products

Rakuten France Multimodal Product Data Classification (ENS Challenge Data 35; also on Kaggle)

Prepared by
Sandeep Grover, Jonathan Vints, Thomas Maisch
Data Scientist project team

Date: 20 July 2026

Interactive project website: https://rakuten-liora-app.streamlit.app

Contents

1. Introduction

1.1 Context

Cataloguing products from their text and image is a core problem for any e-commerce marketplace: the correct category drives search, filtering, recommendation and merchandising. On Rakuten France, millions of third-party sellers upload products with a short title (designation), an optional free-text description, and a single product image. Assigning each listing to the correct product-type code by hand does not scale, and seller-supplied labels are unreliable, because titles are free-form and multilingual, many products have no description, and some are mislabelled.

1.2 Objective

The task is to predict the product-type code (prdtypecode) of each listing from its textual and image data. This is the public Rakuten France Multimodal Product Data Classification challenge [1] (ENS Challenge Data 35, also distributed on Kaggle). The full catalogue spans thousands of classes; the benchmark exposes a curated subset of 27 product-type codes covering the main verticals.

Performance is measured with the weighted-F1 score, the challenge's official metric. The F1 score of a single class is the harmonic mean of its precision (of the listings predicted as that class, how many truly are) and its recall (of the listings truly in that class, how many were found). The weighted-F1 then averages the 27 per-class F1 scores, each weighted by the class's share of listings, so it reflects the quality of the catalogue as a whole under the 14.2-to-1 imbalance. Macro-F1, which weights every class equally, is tracked alongside so that rare-class performance stays visible.

1.3 Project framing

This is a team project and this report covers the full pipeline. Sections 1 to 4 present the business framing, the data exploration and the transformation of the raw text into features. Section 5 presents the text models, from classical baselines on those features to a fine-tuned transformer. Sections 6 and 7 present the image branch and the multimodal fusion that combines the two into the final model.

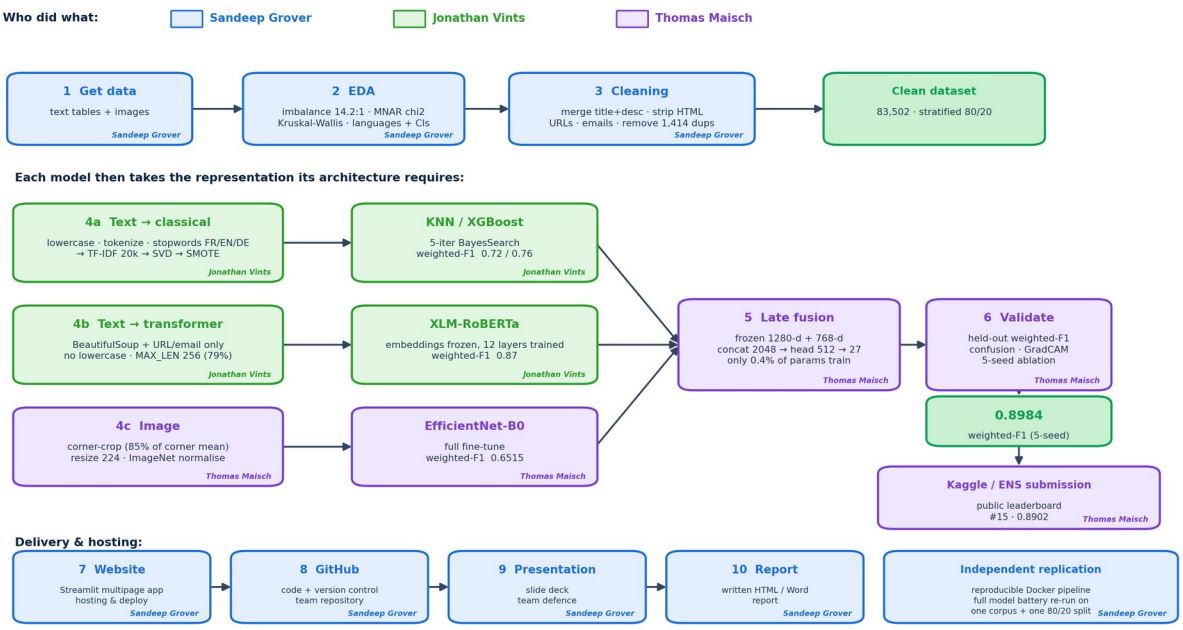


Figure 1. The project pipeline, colour-coded by contributor. A single cleaning pass, duplicate removal and stratified 80/20 split produce one shared dataset. Each model then receives the representation its architecture requires: heavy normalisation and TF-IDF for the bag-of-words classifiers, near-raw text for the transformer, cropped and normalised pixels for the CNN. The fusion loads the two trained branches frozen and trains only the head.

2. Understanding the data

The data come from the Rakuten France challenge. Each listing provides four fields and one image, summarised in the box below.

> **Dataset:** Rakuten France Multimodal Product Data Classification (ENS Challenge Data 35; also on Kaggle).
> **Volume:** 84,916 labelled training listings (83,502 after removing 1,414 exact duplicates); 13,812 unlabelled test listings. Text approximately 60 MB; images approximately 2.2 GB of 500 by 500 RGB JPEGs.
> **Fields:** designation (title, always present); description (optional, HTML-formatted); productid and imageid (image link).
> **Target:** prdtypecode, one of 27 product-type codes.

A duplicate analysis was also run: 1,414 listings (1.7%) were found to be exact duplicates, sharing the same designation and description, and were removed, leaving the 83,502 listings analysed in this report. A further 2,651 (3.1%) share the same title alone but differ in their description, and are retained.

Table 1. The 27 product-type codes and their category names, used throughout the figures.

Code	Category	Code	Category
10	Books (used)	40	Imported games
50	Gaming accessories	60	Consoles
1140	Figurines	1160	Trading cards

1180	Trading figures	1280	Toys
1281	Board games	1300	Model making
1301	Baby textile	1302	Outdoor games/toys
1320	Childcare/baby	1560	Home furnishing
1920	Bedding/linen	1940	Food/grocery
2060	Wall decor	2220	Pet supplies
2280	Magazines	2403	Books/mags lots
2462	Console games (used)	2522	Stationery
2582	Garden furniture	2583	Pool accessories
2585	DIY/garden tools	2705	Books
2905	PC games (dl)		

Note. Names follow the exploratory analysis of section 3; confusion matrices later in the report label classes by these codes.

Because the categories are strongly imbalanced and multilingual, the exploration below quantifies four properties of the data, each of which drives a concrete pre-processing or modelling decision. Every statistical test in this report exists to justify one such decision.

3. Data exploration and visualization

3.1 Class imbalance

The 27 categories are heavily imbalanced: the largest class holds 9,814 listings and the smallest 693, a ratio of 14.2 to 1. A chi-square goodness-of-fit test [2] against a uniform distribution gives a chi-square statistic of 35,282 ($p < 0.001$), so the imbalance is real, not sampling noise (Figure 2). Because the Rakuten challenge is scored on the weighted-F1 score, weighted-F1 is the primary metric; macro-F1 is also tracked, because the imbalance means rare-class performance must be watched. Class weighting and stratified splits are applied.

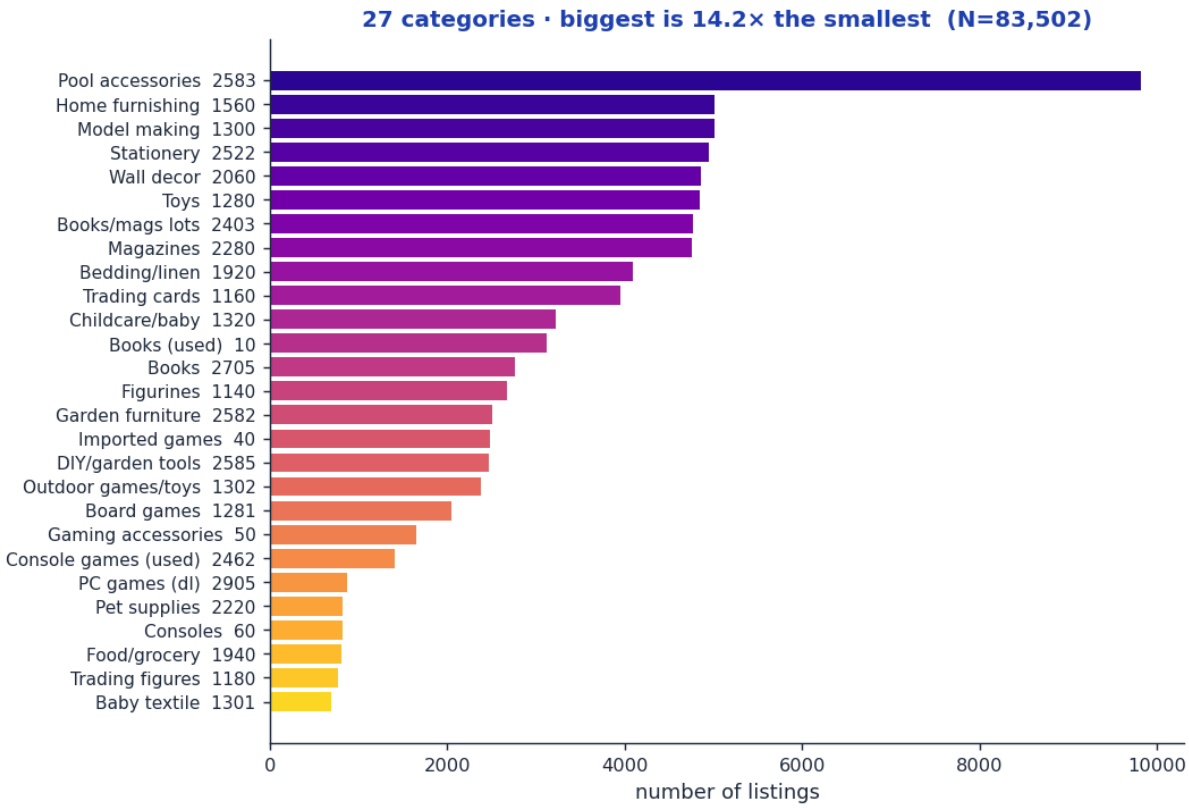


Figure 2. Distribution of the 27 product categories, showing the 14.2-to-1 imbalance.

3.2 Missing descriptions: a Missing-Not-At-Random signal

35.5% of listings have no description. Crucially, the missing rate is not uniform: it varies strongly across categories. A chi-square test of independence between description present or absent and the class gives a chi-square statistic of 42,953 and a Cramer's V of 0.72 [3] ($p < 0.001$), a strong association (Figure 3). The missingness is therefore Missing-Not-At-Random (MNAR) [4]: its very presence is informative. Rather than dropping these rows, which would discard about 35% of the data and hurt exactly the hardest categories, a binary desc_missing indicator is added so the model can use the absence as a feature.

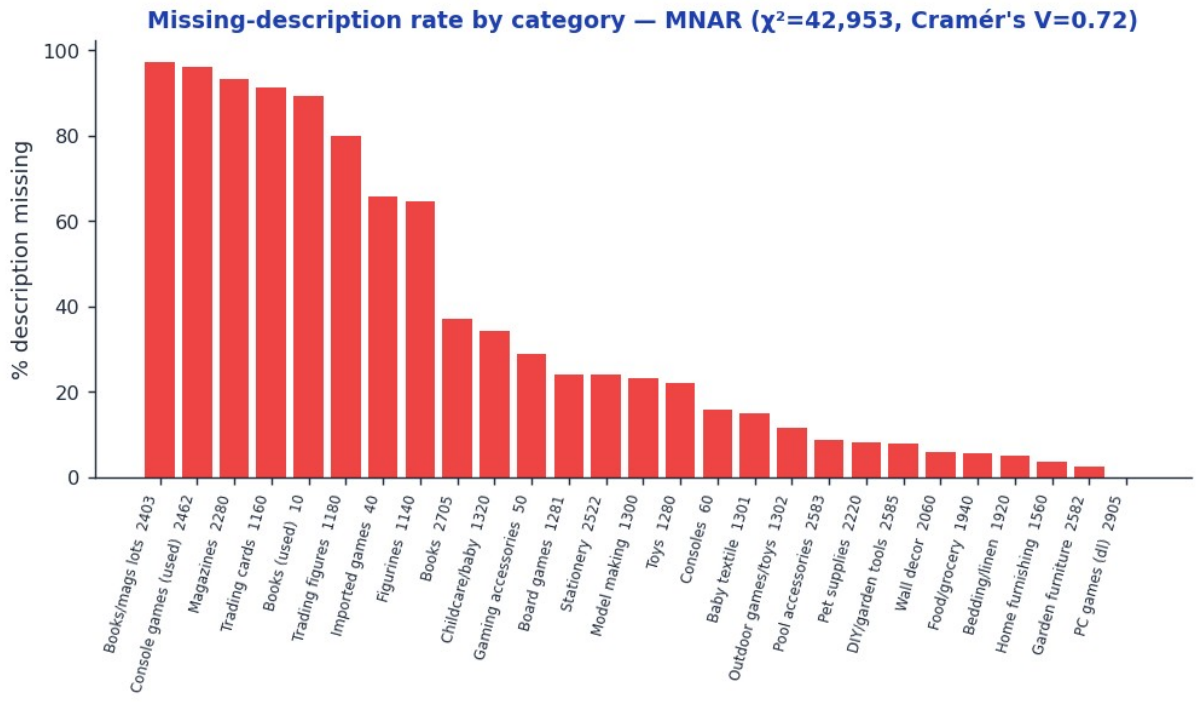


Figure 3. Missing-description rate per category, showing a strong category dependence (MNAR).

3.3 Text length varies by category

Total text length (title plus description) is heavily right-skewed: most listings are short, while a minority run to thousands of characters. Because the distribution is skewed, a non-parametric Kruskal-Wallis test [5] is used instead of a normal ANOVA; it confirms that length differs significantly across categories ($p < 0.001$, Figure 4). The designation and description lengths are therefore retained as numeric features.

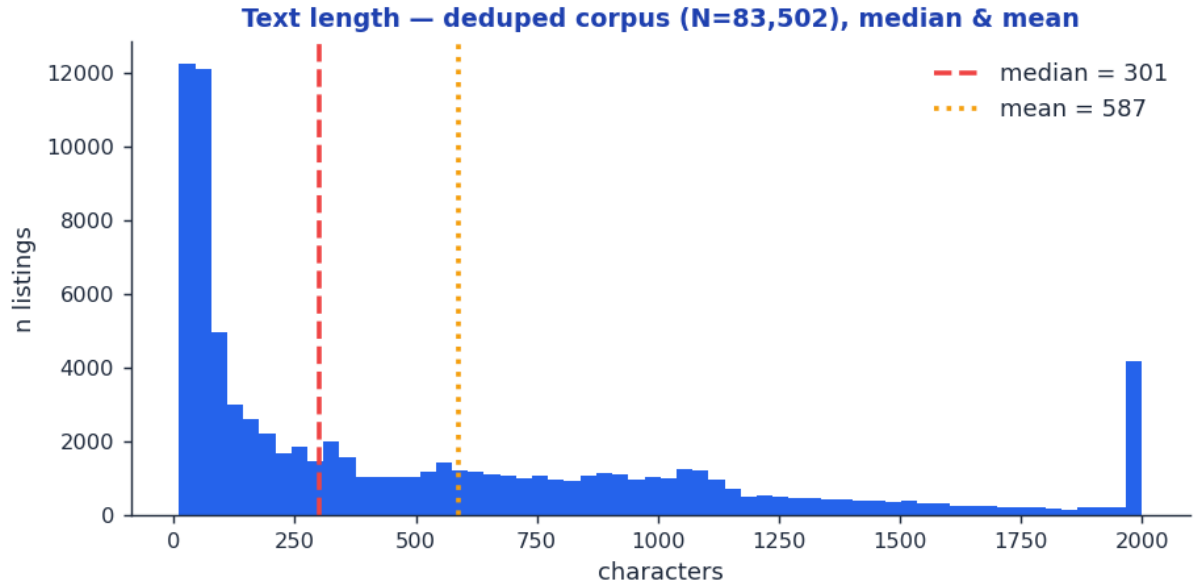


Figure 4. Distribution of text length per listing, with the median marked; note the heavy right tail.

3.4 Language distribution

The text is predominantly French but contains a genuine English and German tail, reflecting sellers from across Europe. Wilson 95% confidence intervals [6], which remain valid for the rare languages where the normal approximation breaks down, confirm that English and German are real sub-populations rather than noise (Figure 5). This favours a French or multilingual language model in the modelling stage, such as CamemBERT (a French RoBERTa) [7], FlauBERT [8], or XLM-R [9].

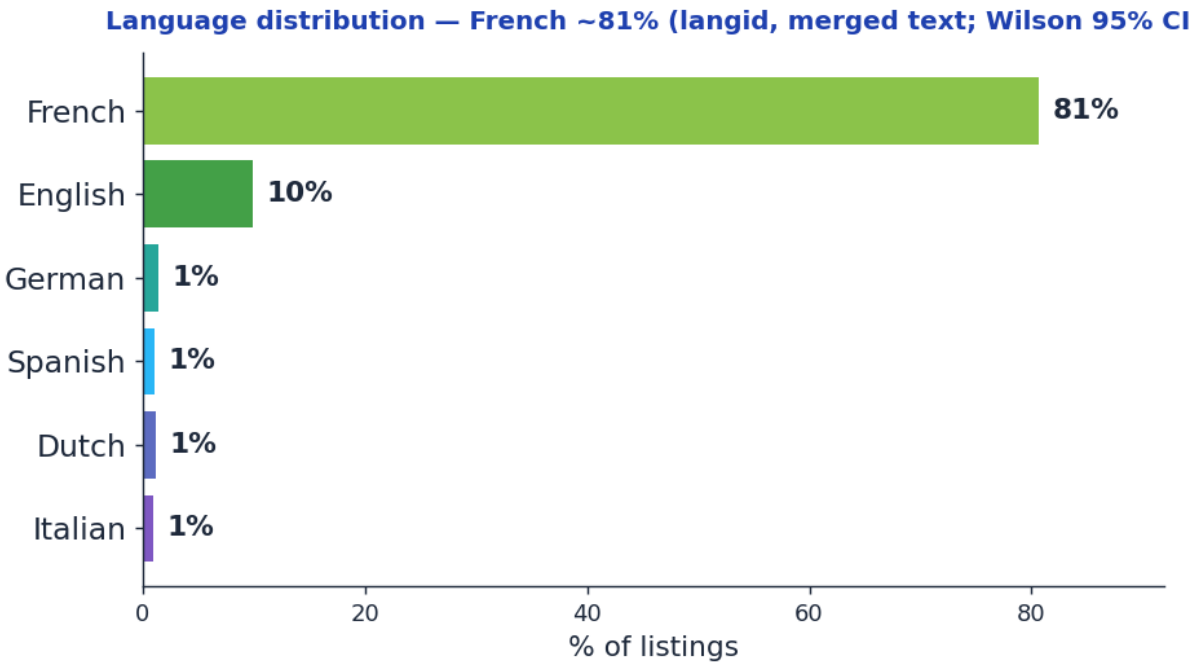


Figure 5. Language distribution across listings: predominantly French, with an English and German tail.

3.5 Word frequency

In the raw text the most frequent words are almost all stopwords, such as de, la, et, pour and les; these appear in nearly every listing and carry no category signal, so they are removed with the NLTK French, English and German stopword lists before vectorisation. Figure 6 therefore shows the twenty most frequent terms after stopword removal, as unigrams and bigrams: they are genuine content words and word-pairs, such as cm, couleur, taille, piscine and eau, which are exactly the discriminative features TF-IDF [10] keeps. At the other extreme, the rarest terms are one-off typos and serial numbers that appear only once; these are removed by the minimum-document-frequency threshold and by capping the vocabulary at the 20,000 most useful terms.

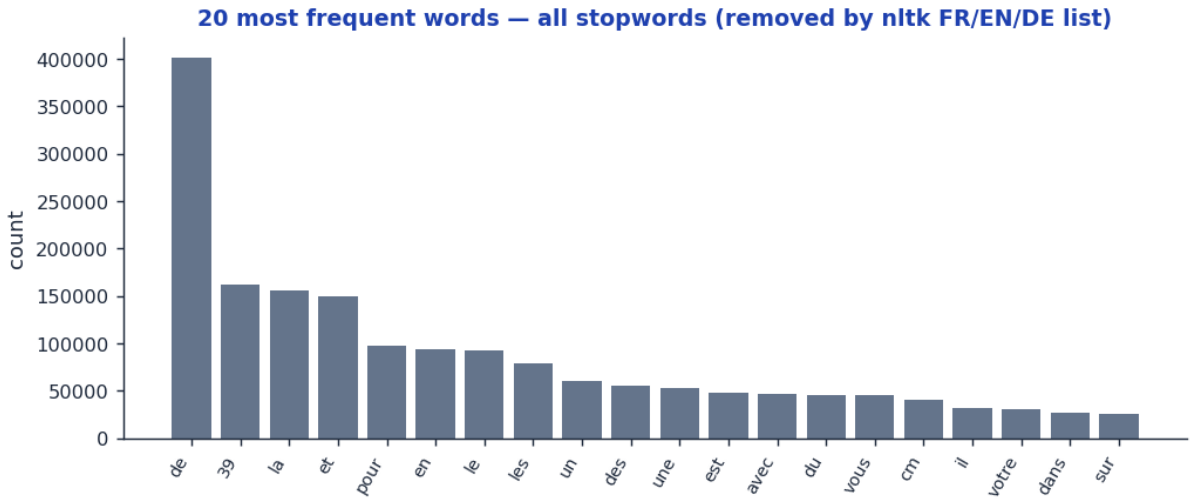


Figure 6. Left: the most frequent raw words are all stopwords, removed by the NLTK French, English and German lists. Right: after removal, the most frequent unigrams and bigrams are content words such as couleur, taille and piscine (bigrams shown in purple).

3.6 Summary of findings

Table 2. Exploratory findings and the pre-processing or modelling recommendation each one drives.

Finding	Key figures	Potential recommendation
Class imbalance	14.2:1 gap; chi-square = 35,282; $p < 0.001$	Weighted-F1 and macro-F1; class weighting; stratified split
Missing descriptions	35.5%; MNAR (chi-square = 42,953, V = 0.72; $p < 0.001$)	Add a desc_missing indicator; do not drop the rows
HTML in descriptions	Present in most non-empty descriptions	Strip with BeautifulSoup before TF-IDF
Text length by class	Kruskal-Wallis; $p < 0.001$	Add designation_len and description_len features
Multilingual content	French ~81%, English ~10%, German ~2% (langid, merged text)	French or multilingual model (CamemBERT, XLM-R)
Text to features	TF-IDF; 20,000 features; 80/20 split	Vectorise merged text; fit on the training set only

Note. Percentages for languages are Wilson 95%-CI point estimates from a labelled sample. Chi-square and Kruskal-Wallis statistics are computed on the full training set of 83,502 listings.

4. Pre-processing and feature engineering

4.1 Text cleaning (per listing, before the split)

The following operations act on each listing independently, so they carry no information between rows and are applied to the whole dataset before splitting. The designation and description are first merged into a single text field and lowercased. URLs, e-mail addresses, HTML tags, HTML entities, size patterns (for

example 8 x 5) and bare numbers are then removed; the HTML is stripped with BeautifulSoup [11], which decodes entities and inserts separators so that a fragment such as rouge
bleu becomes rouge bleu rather than rougebleu, as a naive regular expression would produce. Special characters are removed but French accents are kept, since they are meaningful in French. The cleaned text is tokenised with NLTK's TweetTokenizer, stopwords in French, English and German are removed (the corpus is multilingual), one-character tokens are dropped, and the remaining tokens are stemmed with the French Snowball stemmer for the classical TF-IDF branch; an unstemmed copy is retained for the transformer branch, which needs raw words. Finally, a binary desc_missing indicator is built from the original column, before any fill.

4.2 Train/test split

The data are split 80/20 into training and test sets, stratified on prdtypecode so that both sets preserve the 27-class proportions. The test set is held out purely for evaluation. Any transformation that learns from the corpus is fitted on the training set only, to avoid data leakage.

4.3 TF-IDF vectorization

TF-IDF (implemented with scikit-learn [12]) converts each listing's merged text into a numeric vector [10]. For each term, the term frequency (how often it occurs in the listing) is multiplied by the inverse document frequency, defined as the logarithm of the number of documents divided by the number of documents containing the term. Because stopwords such as de, le and et have already been removed, the vocabulary contains only content words; among these, common words receive a lower weight through the inverse-document-frequency term than rare ones. For example, in our corpus couleur appears in 18,030 documents and so has a low IDF of 2.5, whereas nintendo appears in only 825 documents and has a high IDF of 5.6, giving it far more weight. Both unigrams (single words) and bigrams (adjacent word-pairs) are used, because some category signal lives only in the pair: jeux video (video games) is a strong marker that neither jeux nor video conveys alone. Bigrams greatly enlarge the vocabulary, so it is capped at the 20,000 most useful terms out of approximately 280,000.

The result is a sparse matrix whose rows are the listings and whose columns are the 20,000 vocabulary terms, with each cell a TF-IDF score; approximately 99% of the entries are zero. The vectorizer is fitted on the training set only and then applied to transform both the training and test sets, producing two sparse matrices that share the same vocabulary. The deliverable handed to the modelling stage is therefore a numeric feature set of approximately 66,800 training and 16,700 test rows, comprising the 20,000 sparse TF-IDF features together with the desc_missing and length features, over the 27 target categories.

4.4 Image preprocessing

Most Rakuten photographs place the product inside a large white border. A corner-sampling crop removes it: the average pixel value of the four image corners is taken as the background reference, every pixel darker than 85 percent of that reference is treated as product, the bounding box of those pixels is cropped, and the result is resized to the 224 by 224 network input (Figure 7). Each cropped image path is then attached to the pre-processed dataframe, so every row carries text and image for the same product.

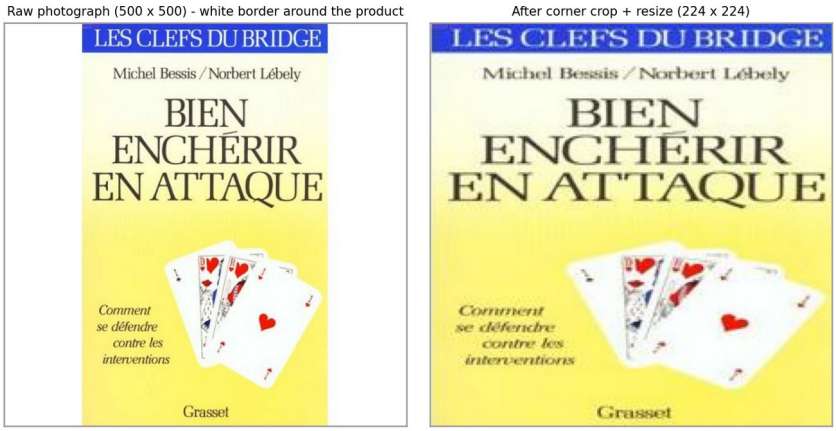


Figure 7. A raw product photograph (left) and the model input after corner-based cropping and resizing to 224 by 224 (right).

5. Text modelling

This stage develops the text-only classifiers along two tracks, and the two deliberately do not share a preprocessing recipe, because they need opposite things from the text. The classical track works on TF-IDF representations compressed with dimensionality reduction, and it is the only track that filters by language: listings whose detected language is French or English are retained, which reduces vocabulary noise for a bag-of-words model that treats every surface form as an independent token. The transformer track uses the full corpus with no language filter, because a multilingual model needs no such routing, and it cleans only minimally, because the surface detail that the classical pipeline deliberately destroys is exactly what self-attention consumes.

5.1 Classical baselines on TF-IDF

The classical pipeline mirrors the section 4 methodology on its filtered corpus: titles and descriptions are merged, HTML and e-mail artefacts removed, the text lowercased and tokenized, and French and English stopwords dropped. A TF-IDF vectorizer is fitted on the training split only, and the sparse matrix is then compressed with truncated SVD [13] into dense components; this compression both speeds up distance computations and helps distance-based models substantially, the K-Nearest-Neighbors classifier [14] in particular. SMOTE [15] balances the training classes by synthesizing minority-class examples in feature space. Two classifier families are tuned with 5-iteration Bayesian hyper-parameter search (BayesSearchCV), scored on the weighted-F1 score: K-Nearest-Neighbors and XGBoost [16]. A representative configuration, a 50,000-term TF-IDF compressed to 1,000 SVD components feeding XGBoost, reaches a weighted-F1 of 0.76; the K-Nearest-Neighbors classifier reaches 0.72. These tuned baselines establish the classical reference the transformer has to beat.

5.2 XLM-RoBERTa classifier

A transformer represents text through self-attention: every subword token weighs every other token to build a context-aware representation of the whole listing, so word order and function words carry signal (the reason stopwords are kept here, unlike in the TF-IDF pipeline). The deep text model is a classifier built on xlm-roberta-base [9], a multilingual RoBERTa [17] pretrained on CommonCrawl text in one hundred languages, loaded and fine-tuned with the HuggingFace Transformers library [18]. The

multilingual vocabulary fits this corpus, which is roughly 81 percent French with a genuine English and German tail (section 3.4), so a single model covers all listings without language routing. The transformer input keeps stopwords, since function words carry the context that self-attention exploits; HTML is removed with BeautifulSoup [11] and e-mail addresses are stripped. The distribution of tokenized lengths was inspected with the model's own tokenizer to set the sequence cap: texts are tokenized to a maximum of 256 subwords, which covers the bulk of listings (median length 301 characters, section 3.3) while keeping memory affordable.

The network is fine-tuned end-to-end. The embedding matrix is frozen, since the subword vocabulary needs no adaptation, while all twelve encoder layers remain trainable so the whole representation can adapt to product language. A new classification head, a dropout of 0.3 applied to the start-of-sequence token followed by a linear layer from the 768-dimensional sentence representation to the 27 classes, replaces the pretraining head; the dropout is there specifically to limit overfitting of the head. Training uses AdamW [19] at a learning rate of 2e-5 with batch size 16 and gradient clipping at 1.0, for up to ten epochs. The learning rate is halved whenever the validation loss stops improving (reduce-on-plateau, factor 0.5, patience 1, floor 1e-7), early stopping watches the validation loss with patience three and a minimum improvement of 1e-4, and the best checkpoint is restored and saved rather than the last.

Two design decisions are worth isolating. First, the sequence cap of 256 subwords is set from the empirical token-length distribution: roughly 79 percent of listings fit under it, and raising it to 512 mostly buys padding rather than content. Second, the text cleaning is deliberately light and its restraint is explicit: designation and description are merged (description is absent about 35 percent of the time, designation never, so both are filled with an empty string first), HTML tags and entities are parsed away with BeautifulSoup [11], and URLs and e-mail addresses are stripped with regular expressions. Nothing else is touched. The text is not lowercased, accents are not folded, digits and punctuation are not removed, and stopwords are not removed, because the model needs them. This is the opposite of the classical branch, and intentionally so: the TF-IDF models treat words as independent tokens, whereas a transformer was pretrained on natural running text and relies on function words and word order for context, so aggressive cleaning destroys the very signal self-attention exploits. The effect is measurable, a weighted-F1 of 0.82 with heavy cleaning against 0.87 with light cleaning, a five-point gap attributable to cleaning alone.

Several adaptations were tested and none improved on this configuration: class weights for the imbalance; keeping designation and description as two separate segments through the tokenizer's native sentence-pair encoding, on the hypothesis that a missing description is itself informative; and a linear-decay learning-rate schedule in place of reduce-on-plateau. The resulting text model reaches a weighted-F1 of 0.87 on held-out data, clearly above the challenge text baseline of 0.8113.

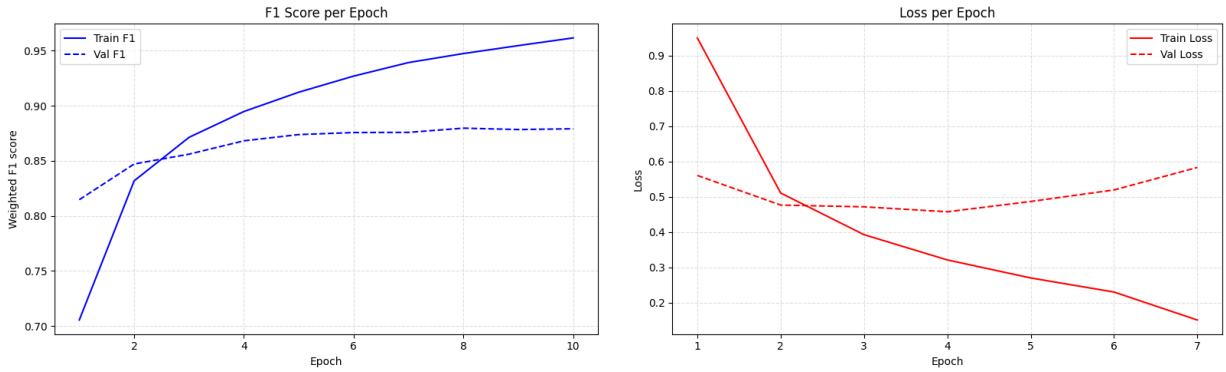


Figure 8. Training the text model, weighted-F1 (left) and cross-entropy loss (right) per epoch. The two panels disagree on when to stop, and that disagreement is the point. Validation loss reaches its minimum at about epoch four and then rises steadily, while validation F1 continues to improve to roughly 0.880 at about epoch eight. The model is becoming more confident on the cases it still gets wrong, which cross-entropy punishes because it scores confidence, while the arg-max decision that F1 measures keeps improving. Selecting on validation loss therefore stops around epoch four and costs about one point of weighted-F1 (0.87 against 0.88) on a task whose official metric is weighted-F1. Validation F1 is already 0.815 after a single epoch, above the challenge text baseline of 0.8113.

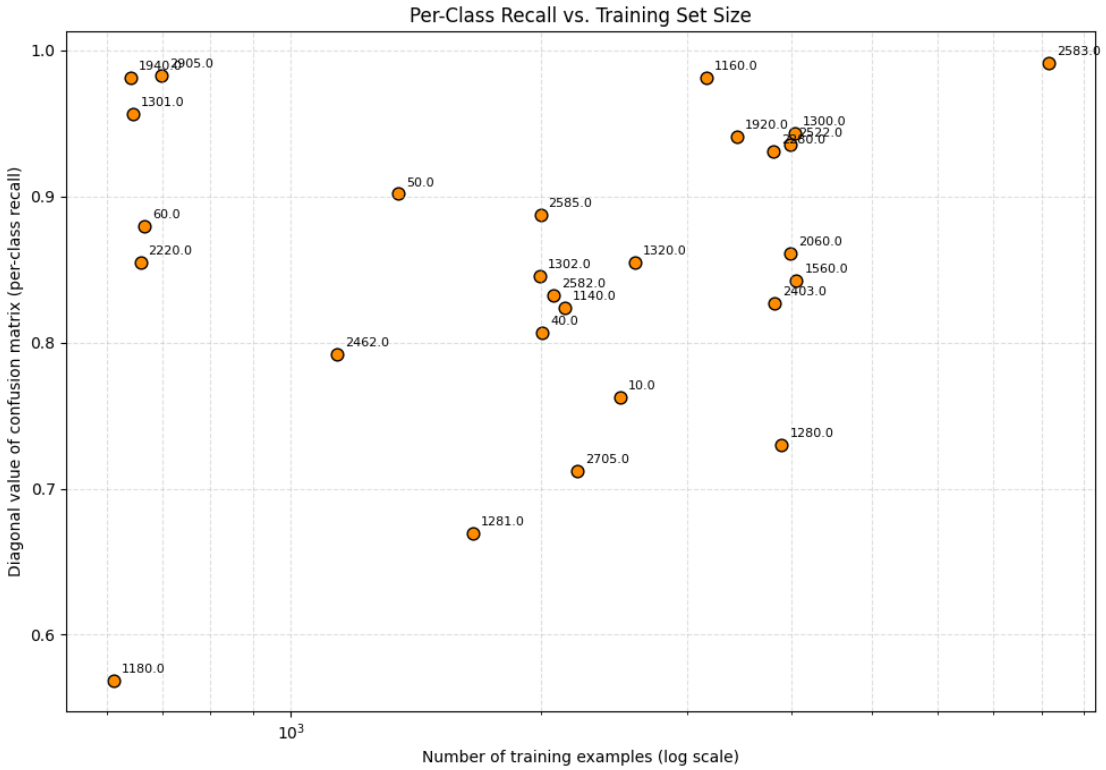


Figure 9. Per-class recall of the text model against the number of training examples available for that class (log scale); each point is one of the 27 categories. Class size alone does not determine performance: several small categories (Food/grocery 1940, PC games 2905, Baby textile 1301) exceed 0.95 recall, while much larger ones (Books 2705, Toys 1280, Board games 1281) sit near 0.67-0.73. The weak categories are the confusable families, not the rare ones.

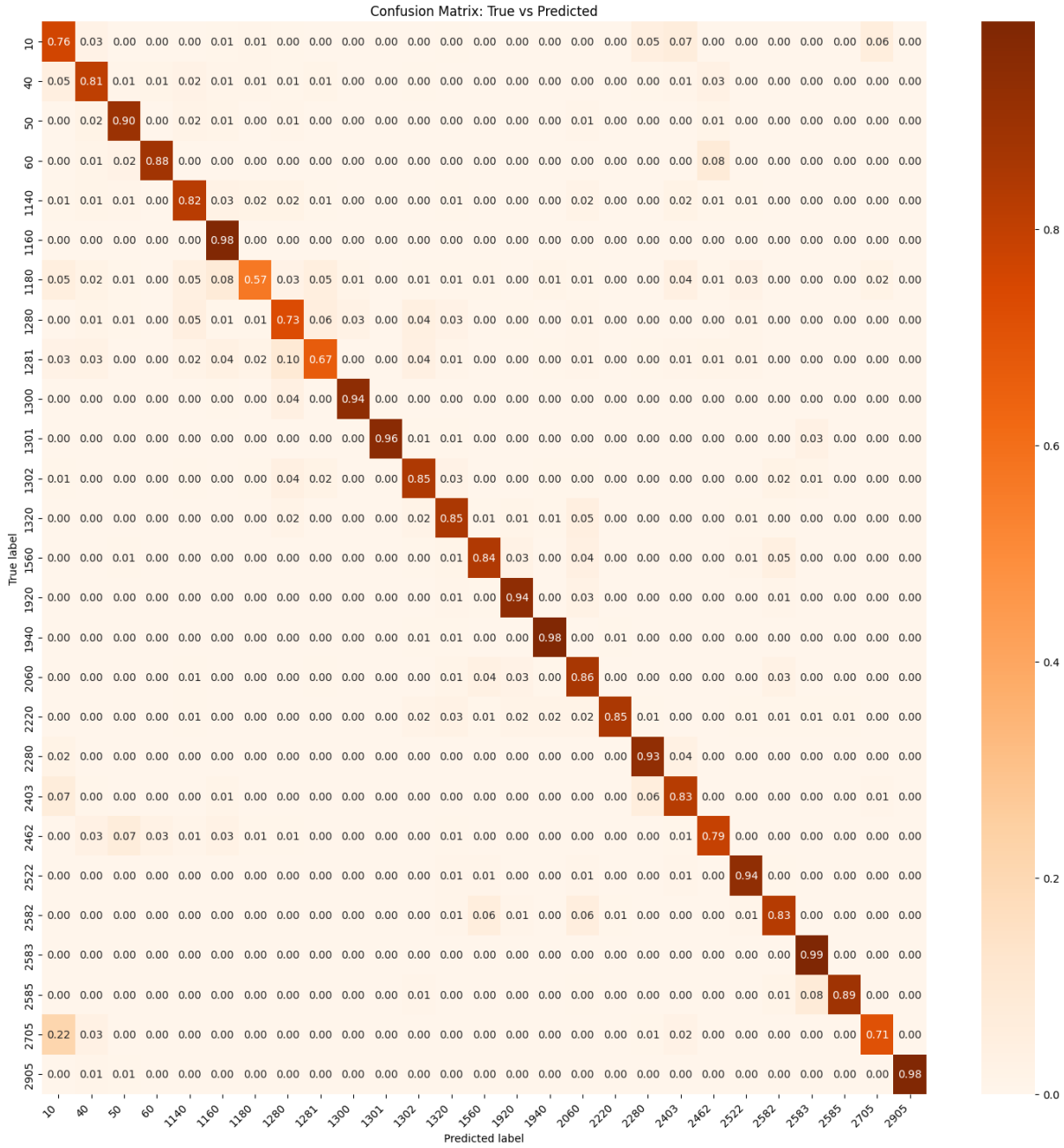


Figure 10. Confusion matrix of the fine-tuned XLM-RoBERTa text model (final configuration, without class weights). The residual errors are concentrated in the same families the image branch struggles with: Trading figures (1180) 0.57, Childcare/baby (1320) 0.65, Board games (1281) 0.67, and 22 percent of Books (2705) predicted as used Books (10).

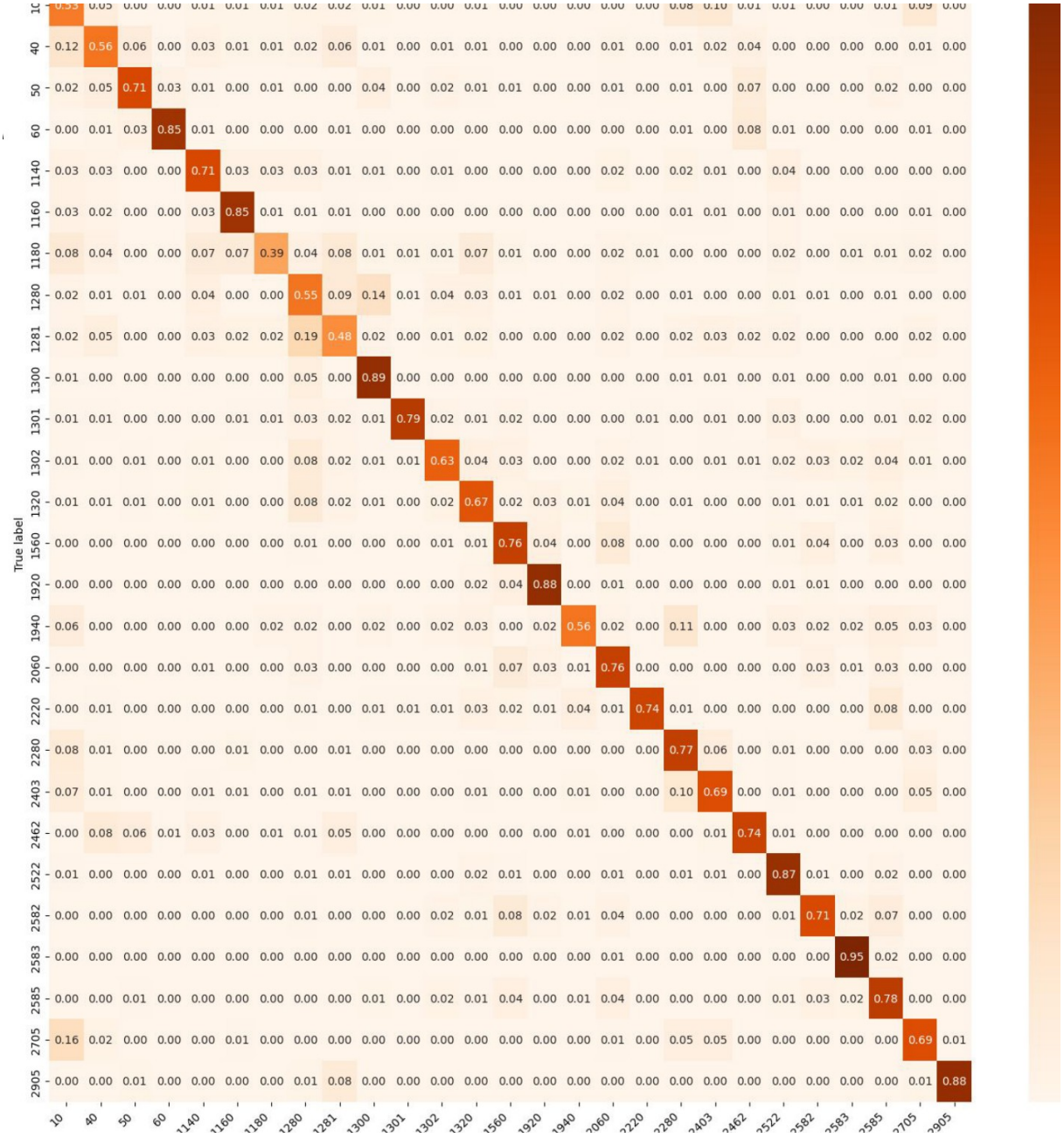


Figure 11. Confusion matrix of the tuned XGBoost baseline on TF-IDF features. Compared with the transformer above, the diagonal is visibly weaker across the board, which is the classical-to-transformer gap expressed per class rather than as a single score.

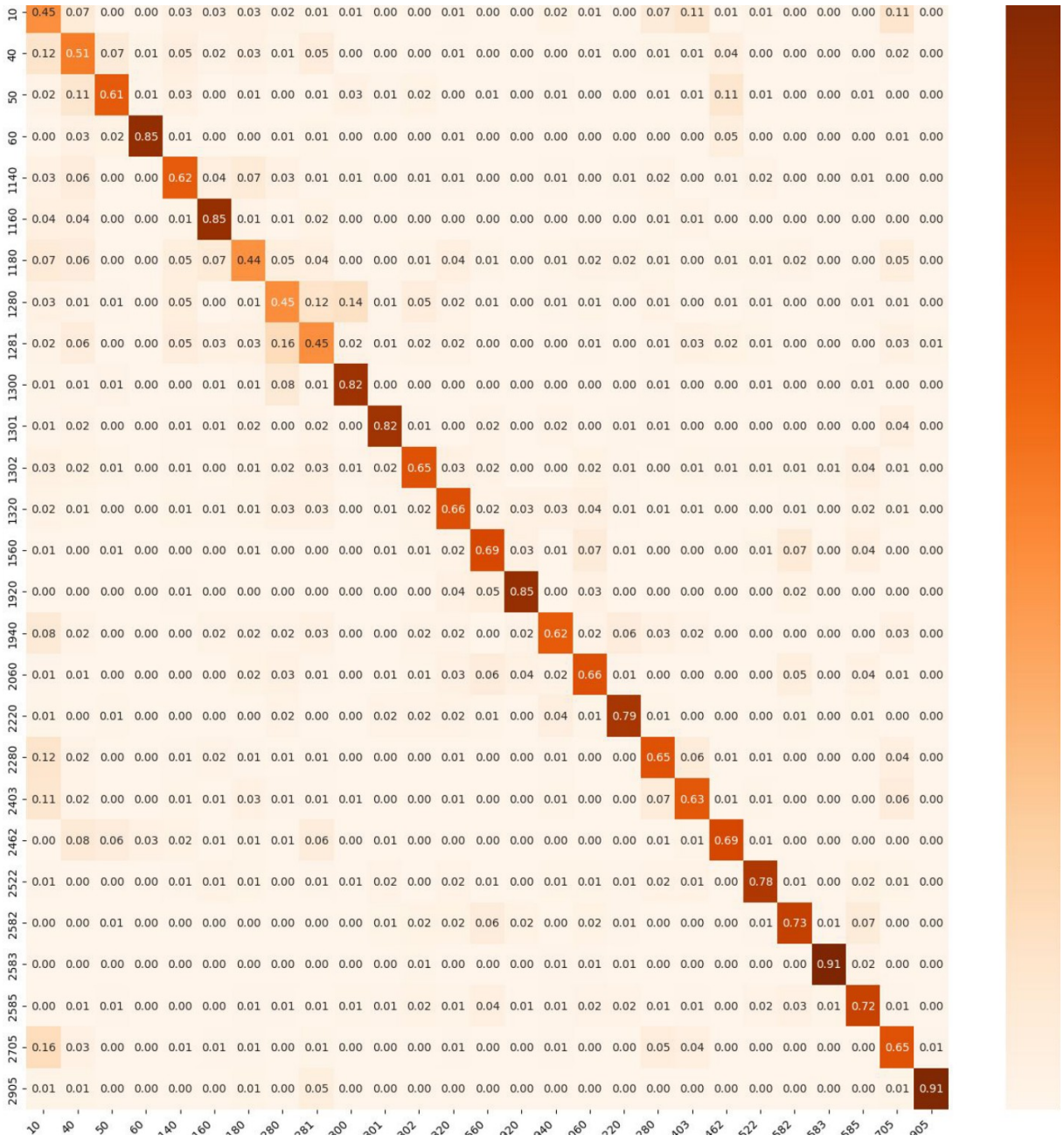


Figure 12. Confusion matrix of the tuned K-Nearest-Neighbors baseline on truncated-SVD features (weighted-F1 0.72). The diagonal is weaker still than the XGBoost baseline, consistent with the 0.72 against 0.76 gap between the two classical models.

5.3 Interpretability and error analysis

Two views make the text classifiers auditable rather than opaque. First, token-level SHAP [20] attributions explain an individual prediction by assigning each subword a signed contribution to the predicted class, so a wrong answer can be traced to the exact words that caused it. On a deliberately hard case, an Xbox decorative sticker skin whose title reads like software, the attributions expose the tokens that pull the model toward the wrong Console games (2462) class instead of the correct accessory category (Figure 13). Second, class weighting was examined per category: it does not raise the overall

weighted-F1, which is why the final configuration keeps weights off, but it does lift recall on the hardest, most confusable classes, trading a little headline accuracy for more balanced coverage (Figure 14).

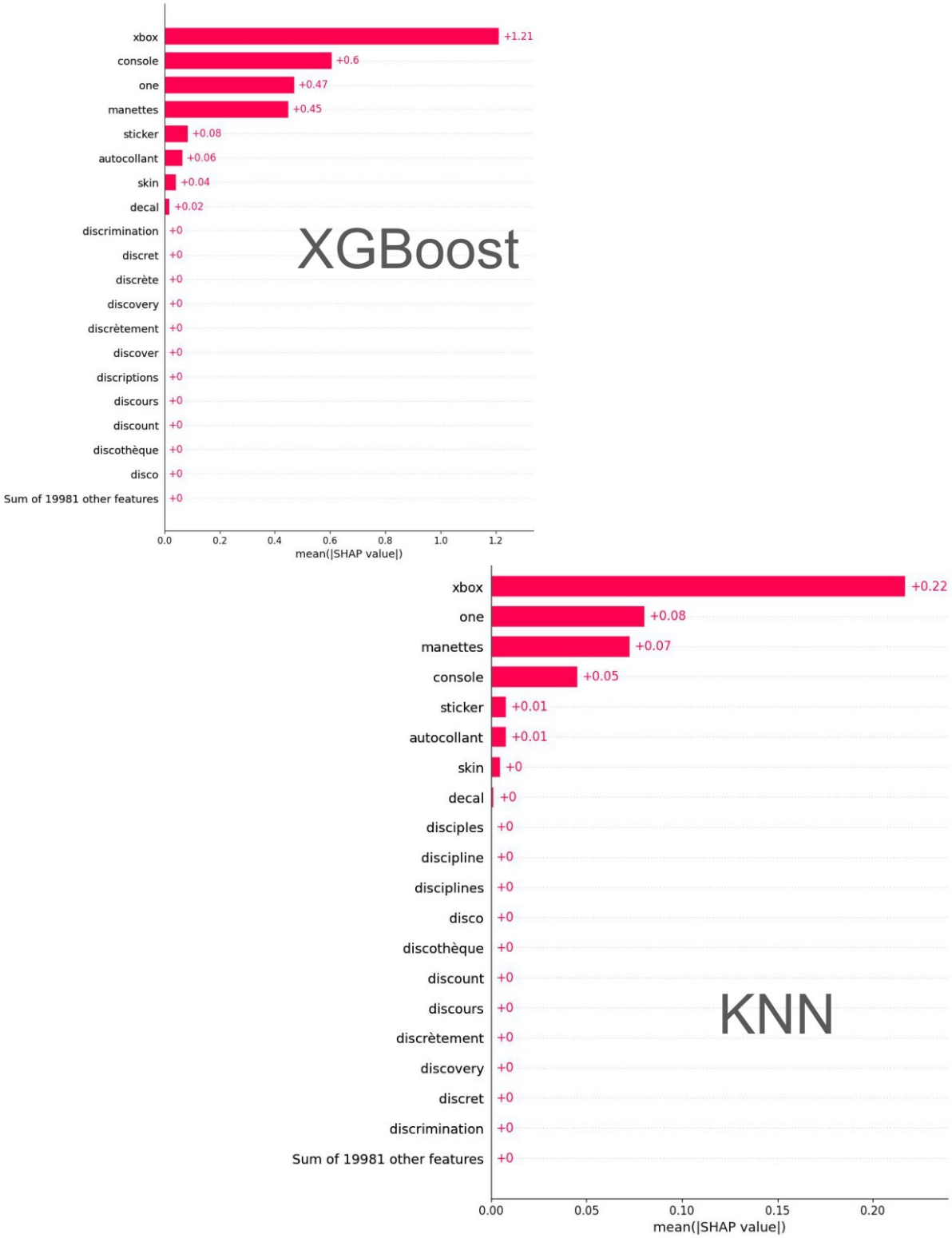


Figure 13. SHAP token attributions for the text model on a hard case: an Xbox sticker skin whose title reads like software and is misread as Console games (2462). The subword tokens are ranked by their signed contribution to the predicted class, so the words that drive the error are visible directly.

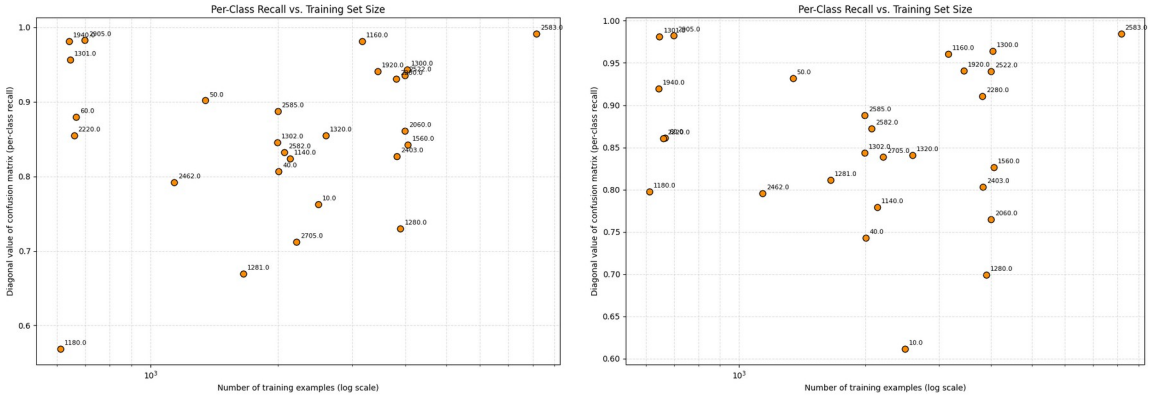


Figure 14. Per-class recall of the text model without versus with class weights. On the hardest, most confusable classes, weighting lifts mean recall (0.87 against 0.86); the overall weighted-F1 is essentially unchanged, which is why the delivered model does not use weights.

6. Image modelling

This stage builds the image branch on the cropped 224 by 224 images prepared in section 4.4. A custom PyTorch dataset wraps the dataframe and feeds shuffled mini-batches of 32 through torchvision transforms with ImageNet normalization; the labels are the same encoded 27 classes as in the text pipeline, and the train/validation split (80/20) is stratified on the class so both sets preserve the imbalance structure. The modelling question is answered in three steps: whether pretraining is necessary at all, which pretrained backbone to adapt and how, and where the resulting model still fails.

6.1 The need for pretraining

A small convolutional network trained from scratch on a 10,000-listing sample reaches a weighted-F1 of only 0.3093, and the standard remedies do not rescue it: data augmentation (0.3063), class weights (0.2932) and a weighted sampler (0.2615) all fail to improve on the baseline (Figure 15). The conclusion is unambiguous: with tens of thousands of images spread over 27 heterogeneous classes, visual features must come from pretraining, and the question becomes which pretrained backbone to adapt.

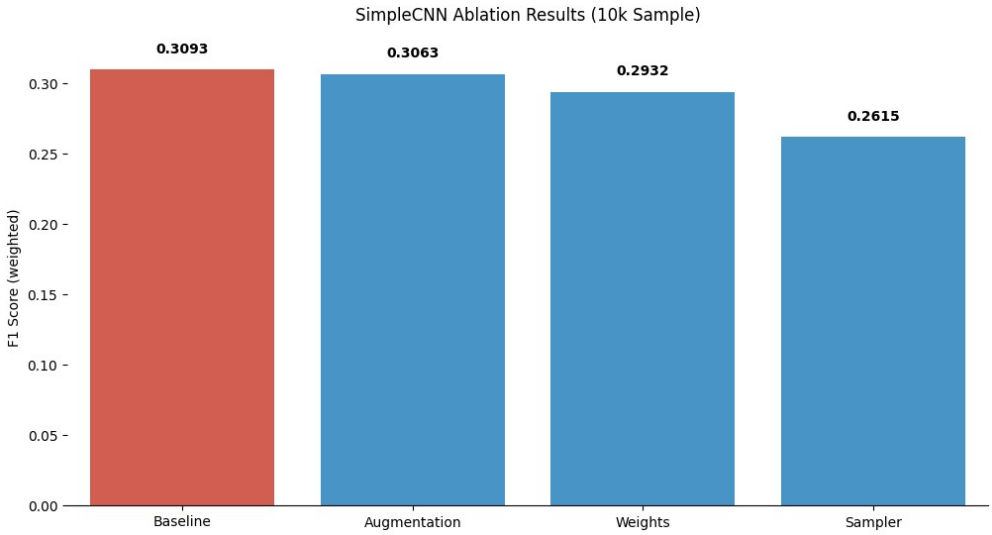


Figure 15. A CNN trained from scratch plateaus near 0.31 weighted-F1; augmentation, class weights and weighted sampling do not help.

6.2 EfficientNet-B0: choice, tuning, result

A convolutional network learns small filters that slide across the image: early layers detect edges and textures, deeper layers compose them into parts and objects, and a final pooled vector feeds the classifier. EfficientNet-B0 [21] is chosen over the challenge's ResNet50 [22] reference on published evidence: it is roughly five times smaller (5.3M against 25.6M parameters) yet more accurate on ImageNet (77.1 against 74.9 percent top-1, Figure 16). Its efficiency comes from depthwise-separable convolutions in inverted-residual blocks with squeeze-and-excitation gating, so most parameters sit in cheap pointwise channel-mixing layers rather than full spatial kernels.

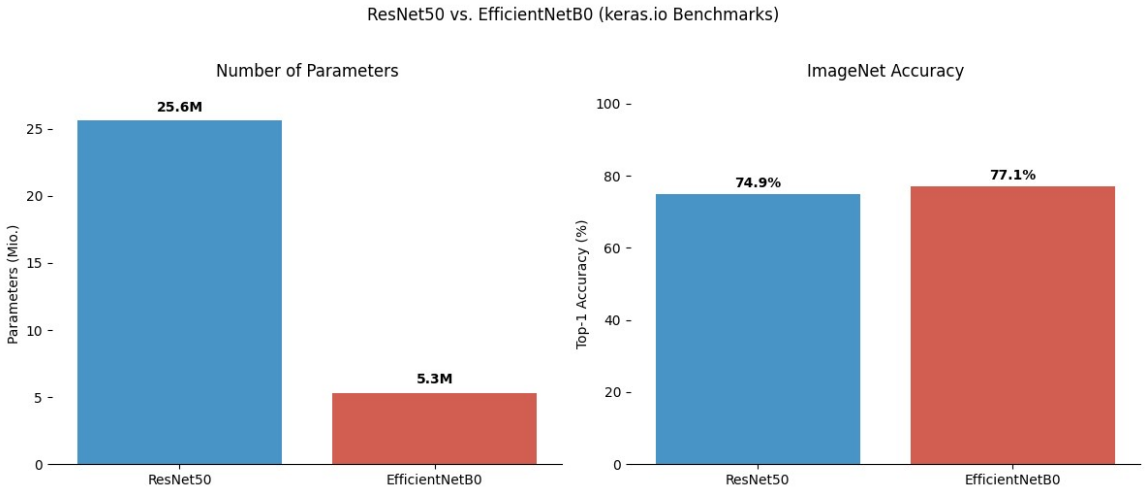


Figure 16. Backbone choice: EfficientNet-B0 is about five times smaller than ResNet50 yet more accurate on ImageNet (keras.io benchmark figures).

An ablation over approximately twenty-five combinations of layer freezing, augmentation, class weights, sampling and classifier design, run on the 10,000-listing sample, lifts the score to 0.5482 (Figure 17) and yields a clear negative finding: freezing hurts. Early layers encode edges and textures that transfer as they are, but the deeper semantic layers are tuned to ImageNet object categories and must adapt to near-

identical product packaging; with roughly 58,000 training images available, full fine-tuning is affordable and superior. The final configuration is therefore fully fine-tuned, with a new two-layer head (1280 to 512 to 27), Adam [23] at a learning rate of 1e-4, up to twenty epochs and early stopping with patience three on the validation loss. Training curves (Figure 18) show the validation F1 plateauing near 0.65 while training F1 keeps rising, the classic overfitting signature that the early stopping intercepts. The final image model reaches a weighted-F1 of 0.6515 on the held-out validation set, above the ResNet50 benchmark of 0.5534.

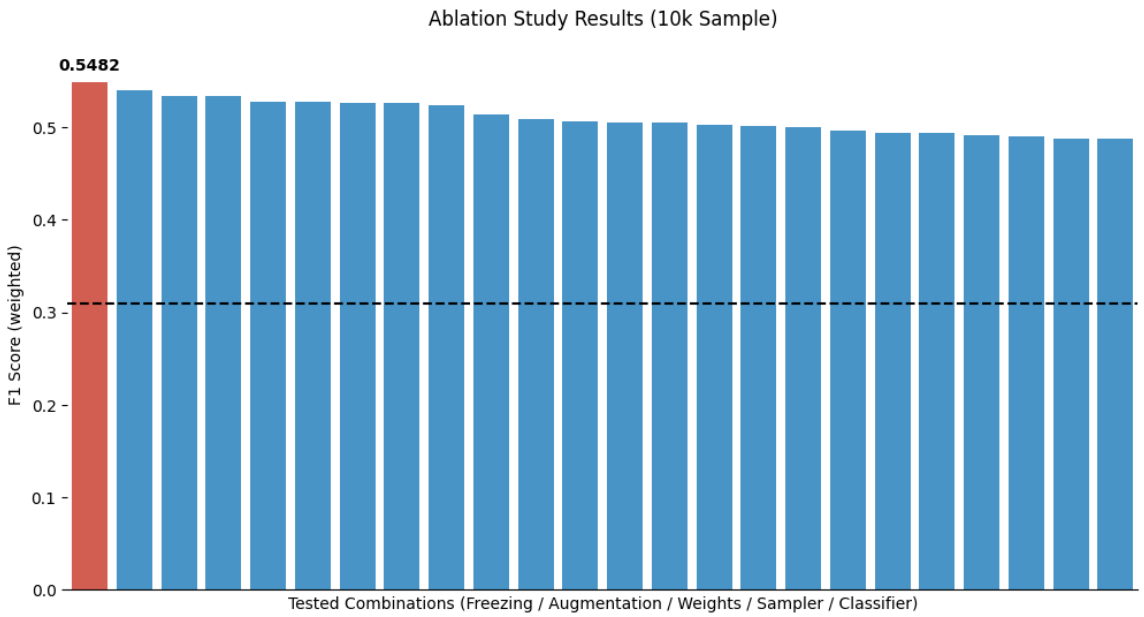


Figure 17. Ablation over about twenty-five freezing / augmentation / weighting / sampling / classifier combinations on the 10,000-listing sample; the best reaches 0.5482, against the 0.31 from-scratch line.

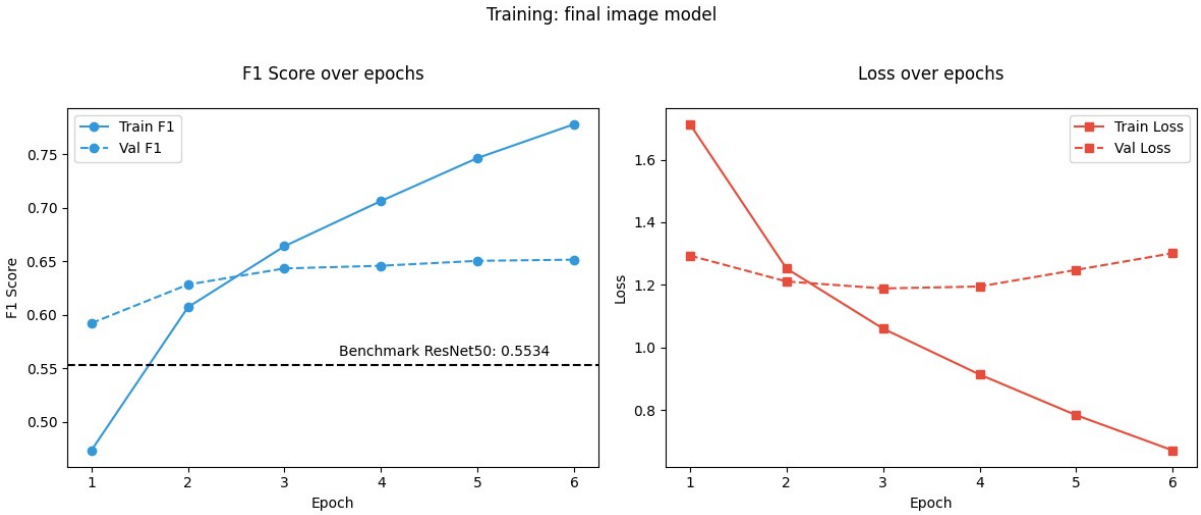


Figure 18. Training the final image model: validation F1 plateaus near 0.65 while training F1 keeps rising; early stopping (patience 3 on validation loss) intercepts the divergence.

6.3 Error analysis and interpretability

A confusion matrix is read row by row: each row is a true class, each column a predicted class, and each cell the share of that row's listings receiving that prediction, so a perfect model is a pure diagonal and every off-diagonal cell is a specific mistake. For the image model the off-diagonal mass concentrates in two visually coherent clusters (Figure 19). Cluster A contains Figurines (1140), Trading figures (1180), Toys (1280), Board games (1281) and Model making (1300): for example 15 percent of Figurines are predicted as Toys, and 20 percent of Board games as Toys. Cluster B contains Books used (10), Magazines (2280), Books/mags lots (2403) and Books (2705), where 16 percent of Books are predicted as used Books. These are families whose members genuinely look alike on a photograph, which is exactly where text disambiguates.

GradCAM [24] explains an individual prediction: it takes the activation maps of the last convolutional layer, weights each map by the gradient of the predicted class score with respect to it, and sums them into a heatmap of the image regions that drove the decision. The maps (Figure 20) confirm the model attends to sensible evidence, focusing on the title band of a book cover and on the object itself for three-dimensional products.

Figure 19. Confusion within the two error clusters of the image model. Cluster A: Figurines (1140), Trading figures (1180), Toys (1280), Board games (1281), Model making (1300). Cluster B: Books used (10), Magazines (2280), Books/mags lots (2403), Books (2705).

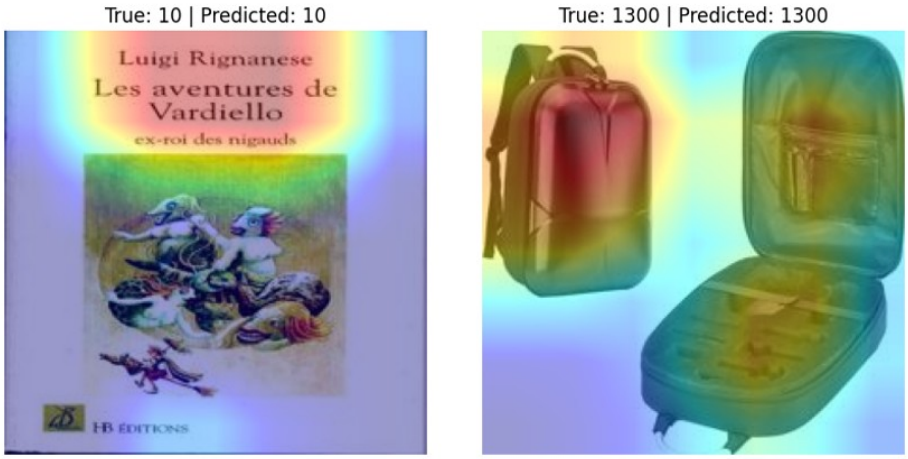


Figure 20. GradCAM activation maps: for a book the network attends to the title band (left); for a case it attends to the object itself (right).

7. Multimodal fusion and final results

The final model combines the two branches by late fusion (Figure 21). It loads the two trained branches as they are: the fine-tuned EfficientNet-B0 image encoder (a 1280-dimensional representation) and the fine-tuned XLM-RoBERTa text encoder (768-dimensional), both frozen. The fusion applies the same text cleaning function as the standalone text model, so the multimodal model sees exactly the same input as the text branch it inherits, and it uses the same stratified 80/20 split as the other tracks; their outputs are concatenated into a 2048-dimensional vector, and only a small fusion head, a linear layer to 512 units with ReLU and dropout followed by a linear layer to the 27 classes, is trained. This head holds 1,062,939 of the network's 283,804,621 parameters, 0.4 percent of the total. Freezing the encoders is the right regime here for the opposite reason that freezing hurt in section 6.2: both branches are already fine-tuned to convergence on this task, so the fusion only needs to learn how to weigh two sets of converged features. Accordingly the head trains at a low learning rate of 2e-5 and reaches its maximum at epoch 9 (Figure 22).

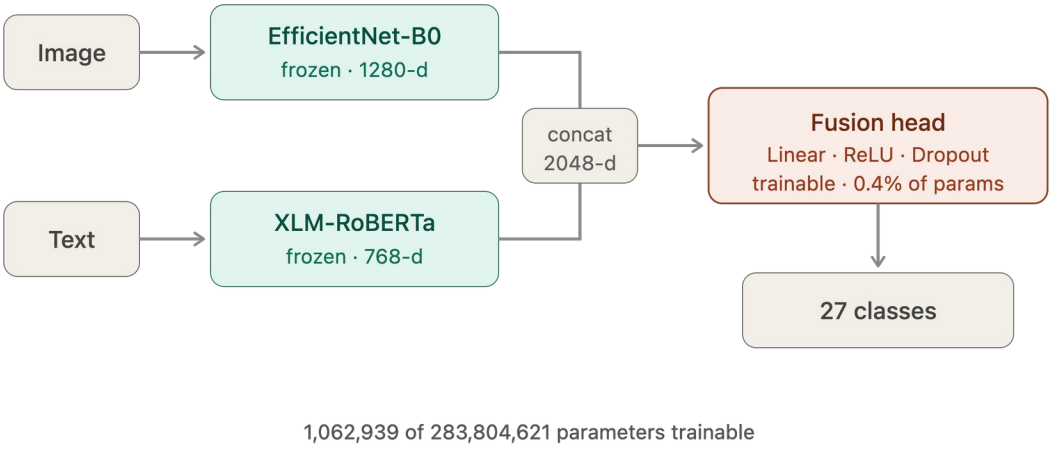


Figure 21. Multimodal architecture: frozen image and text encoders, concatenation to 2048 dimensions, and a small trainable fusion head over the 27 classes.

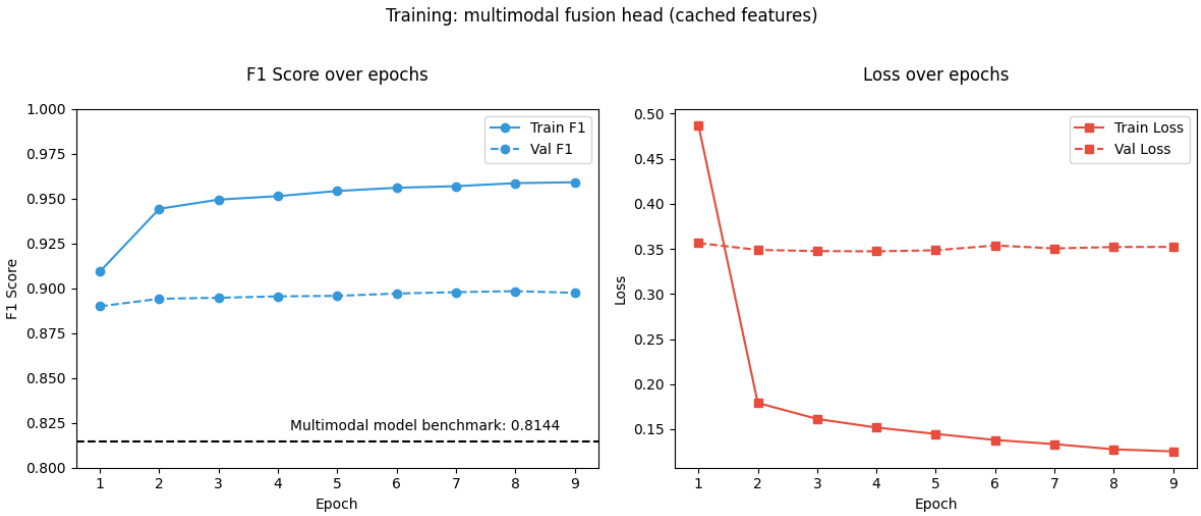


Figure 22. Training the fusion head: validation F1 rises to its maximum at epoch 9; only 0.4 percent of parameters are being trained.

Table 3. Final held-out results (weighted-F1, the challenge's official metric).

Model	Weighted-F1
CNN trained from scratch	0.3093
Challenge baseline: image (ResNet50)	0.5534
EfficientNet-B0, fine-tuned (image model)	0.6515
Challenge baseline: text (title-only CNN)	0.8113
Challenge baseline: official leaderboard entry	0.8144
XLM-RoBERTa, fine-tuned (text model)	0.87

Multimodal late fusion (final model)	0.8984

Note. All scores are validation/test scores, never training scores. The fusion result is confirmed as a mean of 0.8984 over five random seeds. The image and fusion figures are being re-measured on the harmonised 80/20 split and will be updated on completion; the text and baseline figures are unaffected.

The challenge baselines in the table are the reference models published by the challenge organizer, the Rakuten Institute of Technology: a partially unfrozen pretrained ResNet50 for images and a convolutional classifier on the product title alone for text; 0.8144 is the organizer's own entry on the challenge leaderboard. The fused model reaches a weighted-F1 of 0.8984, beating that baseline and every single-modality model, with a clean diagonal across all 27 classes (Figure 23). A five-seed ablation (Figure 24) quantifies what each branch contributes: image-only scores 0.6787, text-only 0.8761 and the full fusion 0.8985. Text therefore carries most of the signal, and the image branch adds a small but consistent gain concentrated exactly where a picture disambiguates: per-class analysis (Figure 25) shows the largest image contributions for Books (2705), used Books (10), Imported games (40) and Books/mags lots (2403), the classes where covers and packaging carry information the text lacks, while a few text-strong categories such as Wall decor (2060) and Toys (1280) are marginally diluted.

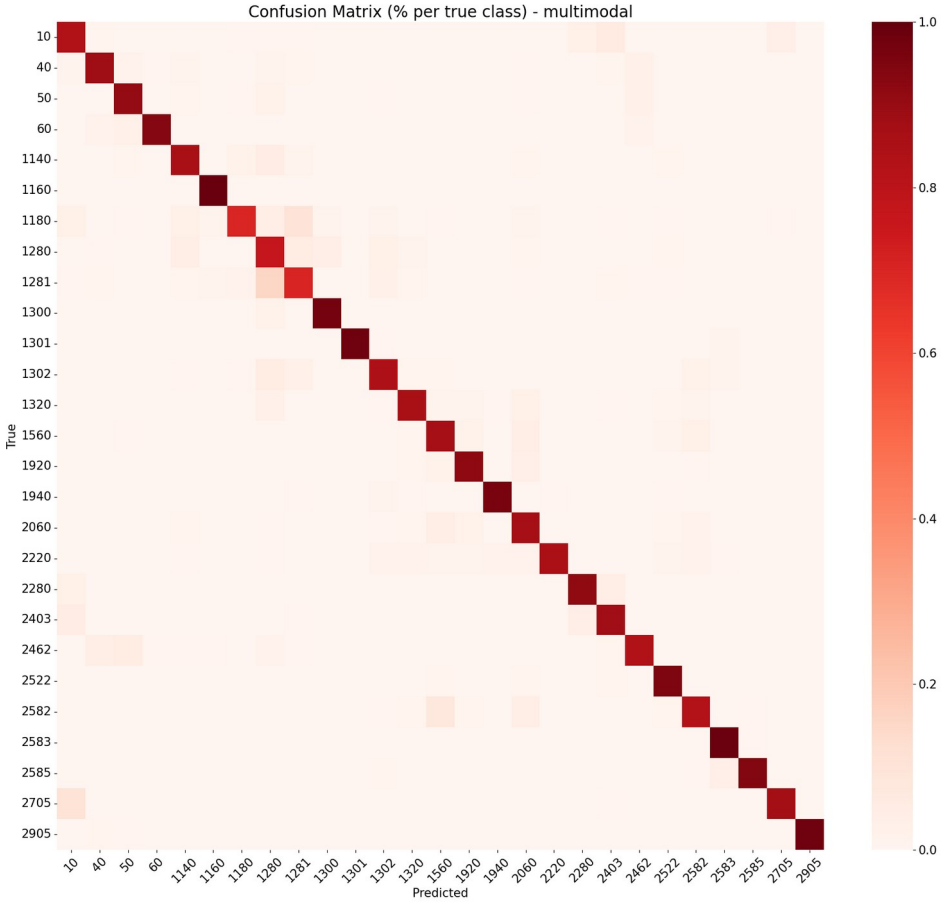


Figure 23. Confusion matrix of the multimodal model (percent per true class): a clean diagonal across the 27 classes.

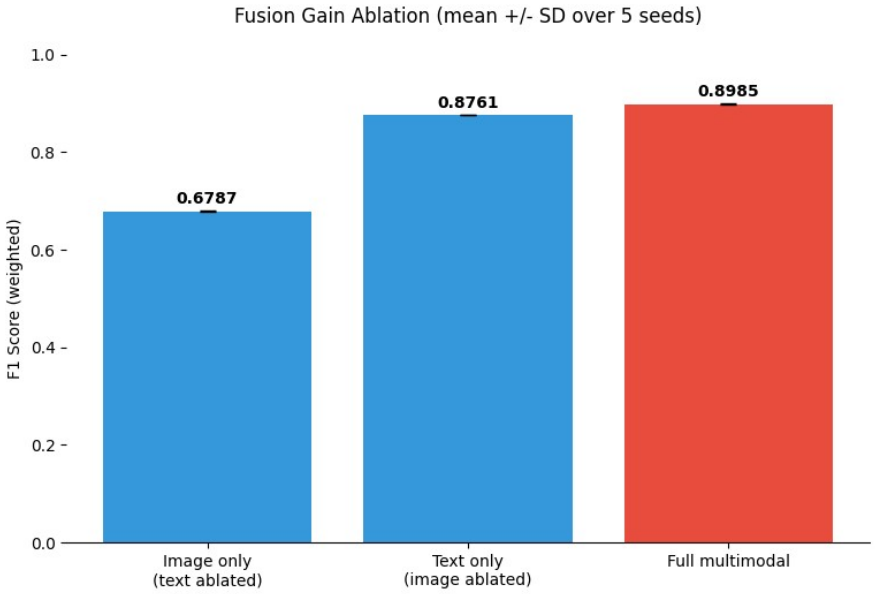


Figure 24. Fusion gain ablation, mean over five seeds: image-only 0.6787, text-only 0.8761, full multimodal 0.8985.

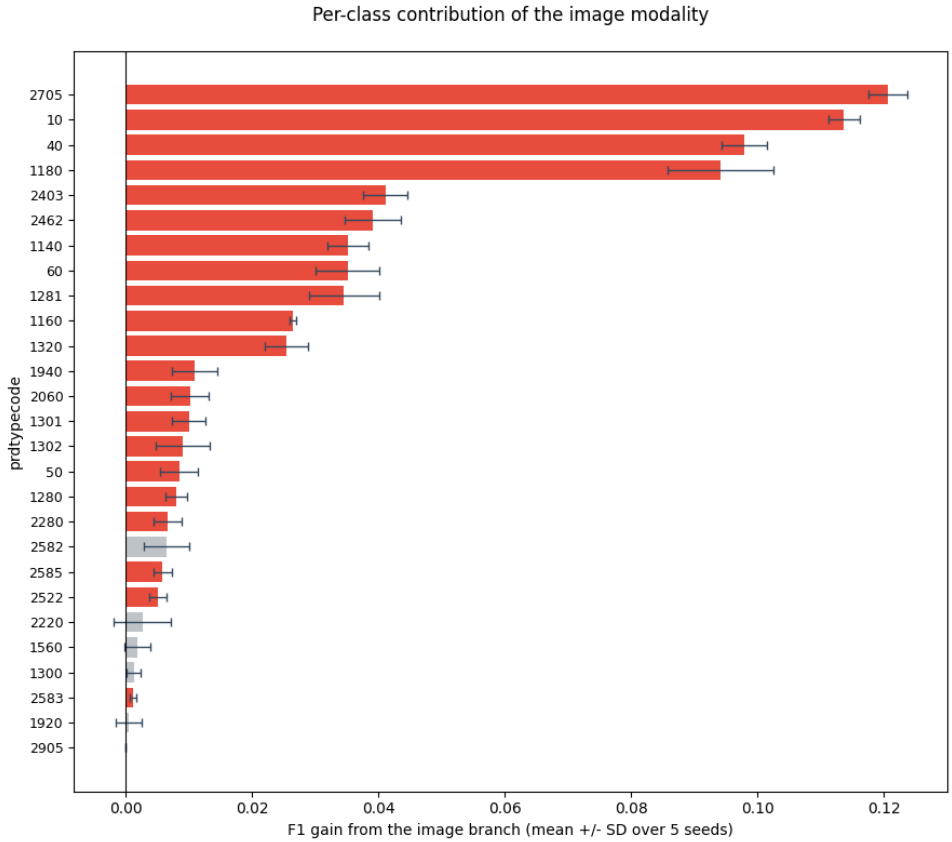


Figure 25. Per-class contribution of the image modality: the image branch helps most where the photograph disambiguates (books, magazines, posters, collectibles).

Two further checks support the design. A cross-attention fusion was also implemented as an alternative to concatenation, projecting the image and text representations into a shared space and letting each attend to the other before classification, which tests whether a richer interaction beats simple concatenation.

Separately, the assumption that the image branch stays frozen was verified rather than assumed, by measuring the drift of its BatchNorm running statistics between the start and the end of fusion training.

The project conclusion is a clean division of labour across the pipeline: the text features of sections 3 and 4 feed both the classical and transformer text models of section 5; the image pipeline of section 6 supplies a complementary signal; and a parameter-efficient late fusion of the two frozen encoders delivers the strongest classifier at 0.8984 weighted-F1. Submitted to the public Rakuten France Multimodal Product Data Classification leaderboard as Sandeep_Jonathan_Thomas_Liora on 16 July 2026, the fused model reached a public score of 0.8902, ranked 15th (Figure 26).

Figure 26. The public leaderboard of the Rakuten France Multimodal Product Data Classification challenge: team Sandeep_Jonathan_Thomas_Liora ranked 15th with a public score of 0.8902.

References

1. Amoualian, H., Goswami, P., Das, P., Montalvo, P., Ach, L., & Dean, N. R. (2021). An E-Commerce Dataset in French for Multi-modal Product Categorization and Cross-Modal Retrieval. In Advances in Information Retrieval (ECIR 2021), LNCS 12657, 18-31. Springer. DOI: 10.1007/978-3-030-72113-8_2.

2. Pearson, K. (1900). On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science, 50(302), 157-175. DOI: 10.1080/14786440009463897.

3. Cramer, H. (1946). Mathematical Methods of Statistics (Princeton Mathematical Series, vol. 9). Princeton University Press.

4. Little, R. J. A., & Rubin, D. B. (2019). Statistical Analysis with Missing Data (3rd ed.). Wiley. DOI: 10.1002/9781119482260.

5. Kruskal, W. H., & Wallis, W. A. (1952). Use of ranks in one-criterion variance analysis. Journal of the American Statistical Association, 47(260), 583-621. DOI: 10.1080/01621459.1952.10483441.

6. Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association, 22(158), 209-212. DOI: 10.1080/01621459.1927.10502953.

7. Martin, L., Muller, B., Ortiz Suarez, P. J., Dupont, Y., Romary, L., de la Clergerie, E. V., Seddah, D., & Sagot, B. (2020). CamemBERT: a Tasty French Language Model. In Proceedings of ACL 2020, 7203-7219. DOI: 10.18653/v1/2020.acl-main.645.

8. Le, H., Vial, L., Frej, J., Segonne, V., Coavoux, M., Lecouteux, B., Allauzen, A., Crabbe, B., Besacier, L., & Schwab, D. (2020). FlauBERT: Unsupervised Language Model Pre-training for French. In Proceedings of LREC 2020, 2479-2490.

9. Conneau, A., Khandelwal, K., Goyal, N., et al. (2020). Unsupervised Cross-lingual Representation Learning at Scale. In Proceedings of ACL 2020, 8440-8451. DOI: 10.18653/v1/2020.acl-main.747.

10. Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1), 11-21. DOI: 10.1108/eb026526.

11. Richardson, L. (2007). Beautiful Soup [Python library, HTML/XML parser]. https://www.crummy.com/software/BeautifulSoup/.

12. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

13. Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by latent semantic analysis. Journal of the American Society for Information Science, 41(6), 391-407. DOI: 10.1002/(SICI)1097-4571(199009)41:6<391::AID-ASI1>3.0.CO;2-9.

14. Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification. IEEE Transactions on Information Theory, 13(1), 21-27. DOI: 10.1109/TIT.1967.1053964.

15. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357. DOI: 10.1613/jair.953.

16. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In Proceedings of KDD 2016, 785-794. DOI: 10.1145/2939672.2939785.

17. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.

18. Wolf, T., Debut, L., Sanh, V., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. In Proceedings of EMNLP 2020: System Demonstrations, 38-45. DOI: 10.18653/v1/2020.emnlp-demos.6.

19. Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. In Proceedings of ICLR 2019.

20. Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. In Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 4765-4774.

21. Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In Proceedings of ICML 2019, PMLR 97, 6105-6114.

22. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. In Proceedings of CVPR 2016, 770-778. DOI: 10.1109/CVPR.2016.90.

23. Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. In Proceedings of ICLR 2015.

24. Selvaraju, R. R., Cogswell, M., Das, A., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. In Proceedings of ICCV 2017, 618-626. DOI: 10.1109/ICCV.2017.74.

Appendix A. Glossary of terms

Table 4. Glossary of the technical terms used in this report.

Term	Meaning
prdtypecode	The target: Rakuten's product-type code, one of 27 categories.
Precision / recall	Of the listings predicted as class X, how many truly are (precision); of the true X listings, how many were found (recall).
F1 score	Harmonic mean of precision and recall for one class.
Weighted-F1	Average of the 27 per-class F1 scores weighted by class size; the challenge's official metric.
Macro-F1	Average of the 27 per-class F1 scores with every class counted equally; keeps rare classes visible.
Confusion matrix	Table of true class (rows) versus predicted class (columns); the diagonal is correct, every off-diagonal cell a specific mistake.
Chi-square test	Statistical test of whether observed counts deviate from an expected distribution more than chance allows.
Cramer's V	Effect size for chi-square association, 0 (none) to 1 (perfect).
MNAR	Missing Not At Random: the fact that a value is missing is itself informative.
Kruskal-Wallis test	Non-parametric test that a numeric quantity differs across groups; used because text length is heavily skewed.
Wilson interval	Confidence interval for a proportion that stays valid for rare events.
Tokenization	Splitting text into units (words or subwords) a model can process.
Stopwords	Very frequent function words (de, le, the); removed for TF-IDF, kept for transformers.
Stemming	Reducing words to their root (chaussures to chaussur); used only in the classical branch.
TF-IDF	Term frequency times inverse document frequency: weights words that are frequent in a listing but rare in the corpus.
n-gram	Sequence of n adjacent words; bigrams capture pairs such as jeux video.
Truncated SVD	Compression of the sparse TF-IDF matrix into a few hundred dense components.

SMOTE	Synthetic Minority Over-sampling: creates synthetic examples of rare classes in feature space.
BayesSearchCV	Bayesian hyper-parameter search: tries configurations informed by previous results.
Transformer / self-attention	Architecture in which every token weighs every other token to build context-aware representations.
CNN / convolution	Network that slides small filters over an image; parameters depend on filter size, not image size.
Transfer learning	Starting from a model pretrained on a large corpus and adapting it to the task.
Fine-tuning	Continuing training of a pretrained model on task data; full (all weights) or partial (some layers frozen).
Frozen encoder	A pretrained network used as a fixed feature extractor; its weights do not change.
Late fusion	Combining the outputs of separately trained branches (here: concatenating image and text features) with a small trained head.
Early stopping	Halting training when validation performance stops improving, to prevent overfitting.
Stratified split	Train/test split that preserves the class proportions in both sets.
Data leakage	Any flow of test information into training; prevented by fitting every transformation on the training set only.
GradCAM	Gradient-weighted class activation map: a heatmap of the image regions that drove a CNN's prediction.

Appendix B. Code, application and live demo

B.1 Code repositories

The team repository holds the shared pipeline and each member's development branch: github.com/DataScientest-Studio/apr26_bds_int_rakuten (branches: Sandeep-preprocessing for the EDA and feature pipeline; Jonathan for the text-model scripts Rakutan_feature_engineering.py and Rakutan_nn.py; Thomas for the image notebooks and the trained EfficientNet-B0 in models/best_model.pth). The consolidated project - this report, the presentation, the interactive application and all figures - lives in github.com/Sandyyy123/rakuten-liora-multimodal.

B.2 Interactive application

The project is delivered as an interactive Streamlit application at rakuten-liora-app.streamlit.app, one page per stage: Home, Data and EDA, Text modelling, Image modelling, Merge and comparison, Run the

model, Report, Presentation, and Q&A. The Home page (Figure 27) summarises the pipeline and the final results.

Liora
ex DataScientest

Liora MLE · Project 06 · Team project

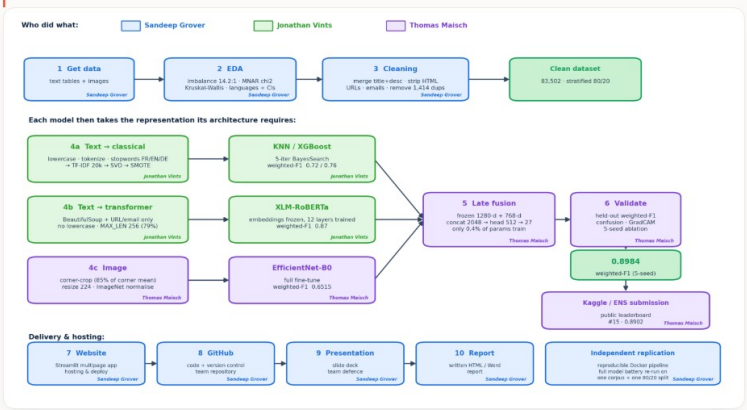
Rakuten France - Multimodal Product Classification

Sorting 84,916 products into 27 categories from their French titles and photos

A team project on the public **Rakuten France** multimodal classification challenge - predict each product's category (`prdtypecode`) from its **text** and its **image**. This app is an interactive walkthrough of the project, one page per stage.

84,916	27	14.2×	35.5%	~81%
PRODUCTS	CATEGORIES	CLASS IMBALANCE	MISSING DESCRIPTION	FRENCH TEXT

How it works - and who did what



The full project pipeline, colour-coded by contributor: **Sandeep Grover** (data, EDA, text preprocessing, website, presentation, report, independent replication), **Jonathan Vints** (classical and transformer text models), **Thomas Maisch** (image model, multimodal fusion, validation and the challenge submission → public leaderboard #15, 0.8902).

Final result

0.8984	#15	0.87	0.6515	0.8144
MULTIMODAL WEIGHTED-F1	PUBLIC LEADERBOARD (0.8902)	TEXT (XLM-ROBERTA)	IMAGE (EFFICIENTNET-B0)	BENCHMARK TO BEAT

Late fusion of the text and image branches is the team's strongest model - see Merge & comparison.

Project sections

- ✅ **Data & EDA** - business case, exploration & text pre-processing → TF-IDF *(Sandeep Grover)*
- ✅ **Text modelling** - TF-IDF baselines (KNN / XGBoost, BayesSearch) → XLM-RoBERTa · weighted-F1 0.87 *(Jonathan Vints)*
- ✅ **Image processing & modelling** - crop → EfficientNet-B0 · weighted-F1 0.6515 *(Thomas Maisch)*
- ✅ **Merge & model comparison** - fuse **Text + Image** → weighted-F1 0.8984 *(Thomas Maisch)*
- 🔵 **Run the model** - live demo (local or hosted)
- 📄 **Report** - the written project report (PDF / Word)
- ❓ **Q&A** - anticipated examiner questions with in-depth answers

Walkthrough: open **Data & EDA** for the full EDA + TF-IDF pipeline, then follow **Text → Image → Merge** to see how the two branches fuse into the 0.8984 multimodal model.

Liora MLE · Project 06 - Rakuten France multimodal product classification (27 classes). Team: Sandeep Grover · Jonathan Vints · Thomas Maisch.
Dataset 84,916 products (83,502 after removing duplicates). Numbers trace to canonical result files.

Figure 27. The application's Home page: pipeline overview, headline statistics and the final weighted-F1 results.

B.3 Live demo

The Run the model page (Figure 28) is the live demo: a French product title (and/or an image) is classified into one of the 27 categories with top-3 confidences. It runs the full model either locally (free, private, one command) or from a public host, and ships with eight real Rakuten products as one-click examples. The trained image branch (final_image_model.pth, EfficientNet-B0 with the 1280 to 512 to 27 head) is already in hand; the text encoder and fusion head complete the full 0.8984 pipeline.

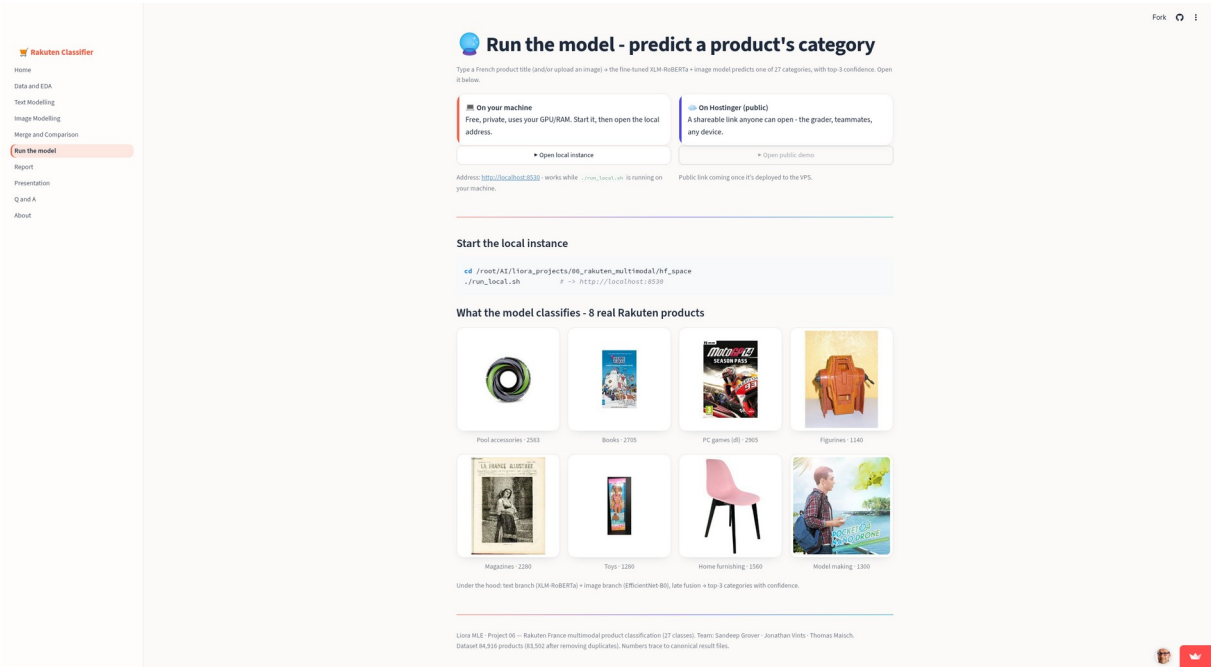


Figure 28. The Run the model page: the live demo with eight real Rakuten example products.